ShadowMem — Frontend Architectural Audit

In-Depth Evaluation: Next.js App Router, State Architecture, SOC Ergonomics, & Backend Alignment

Target Application:	ShadowMem Frontend (Next.js 14.2 / TypeScript / Tailwind CSS / Recharts)	**Audit Timestamp:**	October 11, 2026 - 17:50
Delivery Mode:	Static Export (output: 'export') served via FastAPI (app.py)	**Backend Version:**	FastAPI 5-Step Modernized Architecture (/api/v1)
Current Architecture:	6 App Routes, 12 Custom Components, Local Component State	**OVERALL RATING:**	**GRADE B- (NEEDS MODERNIZATION)**

1. Executive Summary & Audit Context

The ShadowMem frontend was successfully migrated from an embedded monolithic HTML file into a modern **Next.js 14 App Router** project utilizing TypeScript and Tailwind CSS. The production application builds to a static export in frontend/out/ and is hosted directly by the FastAPI backend. While the application presents a functional, cohesive interface with custom radar and analytics visualizations, our deep code audit identified significant architectural debt, backward alignment lags, and state fragmentation that must be resolved to support upcoming capabilities (WSL2 sandbox detonation and supply chain scanning).

Key Findings: (1) *Endpoint Lag:* The frontend communicates entirely with legacy unversioned endpoints (/api/scan, /api/isolate?reason=...) and does not consume the newly verified /api/v1/* schemas. (2) *State Fragmentation:* Forensics telemetry is siloed in local component state; navigating between tabs resets all scan results. (3) *Theme Incongruity:* While expected to reflect a CrowdStrike-style dark navy palette, the active theme is a warm parchment light mode (#FAF7F2) where orange (#E8793C) is confusingly aliased as 'lime'. (4) *WebSocket Duplication:* Up to three redundant WebSocket links are spawned simultaneously, while incoming memory_map frames are actively dropped. (5) *Simulated Artifacts:* Legacy timers continue to generate fake kernel logs on an interval.

2. Frontend Subsystem Health Scorecard

Subsystem Area	Current Implementation State	Deficiencies & Observations	Health Rating
API & Backend Alignment	Uses src/lib/api.ts with relative URLs	Lags behind backend: calls unversioned endpoints (/api/scan), uses query params for isolation, misses /api/v1/* schemas.	**POOR (C)**
State Management	Local useState in each page; 3 light Contexts	No global scan store. Switching tabs (e.g. from Overview to Live Map) erases scan data. Redundant scans required per page.	**FAIR (C+)**
WebSocket Handling	Linear backoff reconnect in api.ts	Multiple concurrent connections spawned (Context + Page); actively discards memory_map telemetry frames.	**FAIR (B-)**
Design & Styling	Tailwind CSS with custom extend palette	Warm parchment theme (#FAF7F2) conflicts with SOC expectations; 'lime' is aliased to orange (#E8793C); no dark mode toggle.	**FAIR (B-)**
Component Architecture	12 feature components in components/dashboard	Zero shared UI primitives (Button, Card, Badge, Modal). Inline Tailwind classes copy-pasted; modal logic duplicated across 4 pages.	**GOOD (B)**
Static Export & Delivery	Next.js output: 'export' served by app.py	Flawless production delivery. Dev proxy has hardcoded port 8000 in next.config.mjs, but production relative paths survive port changes.	**EXCELLENT (A)**
Step 5 Feature Readiness	No sandbox or supply chain routes exist	Completely lacks UI structures, types, or API clients for Docker sandbox or PE supply chain scanner.	**INCOMPLETE (D)**

3. Detailed Technical Audit Findings

A. Backend Alignment: Unversioned Legacy Endpoint Lock-In

While the FastAPI backend was upgraded to versioned /api/v1/* schemas with typed Pydantic request bodies, src/lib/api.ts remains tied to legacy endpoints. Specifically: (1) fetchScan() calls GET /api/scan rather than /api/v1/scan. (2) isolateProcess() encodes justification into the query string (?reason=...) rather than passing a JSON payload (IsolateRequest). (3) AlertsPage executes an unmanaged, inline raw fetch('/simulate/alert'). Fortunately, because getApiBaseUrl() returns an empty string in the browser, all API paths are relative and will survive backend port modifications in production, though dev-mode rewrites in next.config.mjs remain pinned to port 8000.

```
// src/lib/api.ts (lines 59, 92-94) -- Current Legacy Calls
const res = await fetch(`${base}/api/scan`, { method: 'GET' });
const query = reason ? `?reason=${encodeURIComponent(reason)}` : '';
const res = await fetch(`${base}/api/isolate/${pid}${query}`, { method: 'POST' });

// Target Modern Architecture (/api/v1/ with JSON body)
await fetch(`${base}/api/v1/isolate/${pid}`, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ reason }),
});
```

B. State Management Fragmentation & Discarded Telemetry

The frontend lacks a global forensic scan store. processes, summary, hasScanned, and isolatedPids are re-declared locally using useState in five separate page files (app/page.tsx, process-table/page.tsx, live-map/page.tsx, alerts/page.tsx, and analytics/page.tsx). Navigating between dashboard tabs destroys all existing scan state, forcing the analyst to re-trigger a 12-second volatile memory scan each time. Furthermore, in src/lib/api.ts (line 199), incoming WebSocket frames with type !== 'alert' are discarded, meaning the backend's real-time memory_map broadcast is completely ignored by the client.

C. Redundant WebSocket Connections & Artificial Ambient Tickers

Inspection revealed that ConnectionContext.tsx, app/page.tsx, alerts/page.tsx, and kernel-log/page.tsx each independently invoke connectAlertsWebSocket(). When an analyst browses the application, up to three concurrent WebSocket links are held open with the backend. Concurrently, app/page.tsx (lines 142–174) and kernel-log/page.tsx retain legacy demo tickers that execute setInterval loops with Math.random() > 0.45, injecting fabricated kernel strings (e.g. *'Walking active thread list via PsGetProcessThreads...'*). These synthetic logs confuse live incident response and must be retired.

D. Design Token Incongruity: The 'Lime Orange' Dilemma & Dark Mode Absence

The original single-file dashboard was styled with an authentic dark SOC palette (#0A0E1A background, #151B2B cards, and #C6FF3D lime green). During the Next.js rebuild, the theme was converted to a warm light parchment palette (#FAF7F2 background, #FFFFFF cards). In tailwind.config.ts, the color key lime was re-mapped to orange (#E8793C). This produces severe cognitive disconnect: classes named bg-lime render orange elements, and alerts styled with yellow warnings clash with the primary orange branding. Furthermore, while darkMode: 'class' is configured, zero dark: variant classes exist in the codebase, and no theme toggle is present.

4. SOC Tool UI/UX & Information Architecture Audit

UX Dimension	Current State in ShadowMem	SOC Standard & Recommended Remediation
Information Hierarchy	Process table is hidden on a separate page (/process-table). Dashboard shows only aggregate stat cards and recent alerts.	SOC tools (CrowdStrike, Sentinel) prioritize immediate process anomaly visibility on the main console. Add a high-risk process drawer directly to the Overview.
Threat Visual Coding	4 distinct status colors: Red (malicious), Amber (suspicious), Blue (JIT), Emerald (clean). High visual distinction.	Adheres well to MITRE ATT&CK; visual conventions. Ensure color-blind accessible icons (e.g. ShieldAlert vs CheckCircle) accompany every score pill.
State Feedback	Basic spinners on buttons; generic connection banner on disconnect; scan error text banner.	Lacks non-blocking toast alerts, granular retry actions, and partial-isolation progress indicators during multi-thread containment.
Accessibility & Ergonomics	Canvas radar on /live-map only reacts to mouse hover; text contrast is 4.6:1; no keyboard focus rings.	Radar nodes are completely unreachable via keyboard navigation (Tab/Enter). Canvas must be paired with an accessible, keyboard-focusable DOM tree.

5. Step 5 Feature Readiness: Sandbox & Supply Chain Architecture

With the backend Step 5 verification officially certified, the frontend must now ingest two rich data contracts without undergoing another structural rewrite. Below is the proposed component and route architecture informed by the actual backend response schemas:

```
frontend/src/
■■■ types/
■   ■■■ sandbox.ts            # Mirrors SandboxExecutionResult & SandboxHealthResponse
■   ■■■ supply_chain.ts       # Mirrors SupplyChainScanResponse & ModuleIntegrityFinding
■■■ lib/api/
■   ■■■ sandbox.ts            # submitSample(), getJobStatus(uuid), getSandboxHealth()
■   ■■■ supplyChain.ts        # scanProcessModules(pid), scanDirectoryPath(path)
■■■ app/
■   ■■■ sandbox/page.tsx      # Detonation Console (File Uploader, CLI Args, Job Poller)
■   ■■■ supply-chain/page.tsx # Authenticode & Module Integrity Inspector
■■■ components/
    ■■■ sandbox/
    ■   ■■■ DetonationSubmitForm.tsx   # File/Base64 input, arguments, timeout settings
    ■   ■■■ JobStatusTracker.tsx       # Live UUID poller with status badge (disabled/failed/completed)
    ■   ■■■ ExecutionResultViewer.tsx  # Exit code, stdout/stderr console, artifacts drawer
    ■■■ supply-chain/
        ■■■ ModuleIntegrityTable.tsx   # DLL list, signed badge, Authenticode cert, SHA-256 copy
        ■■■ PathScanControls.tsx       # Directory selector with quick targets (.venv, System32)
        ■■■ SupplyChainPostureCard.tsx # Posture pill (Protected, Suspicious, Vulnerable)
```

6. Current vs. Target Architectural Comparison

Architectural Vector	Current State (As-Is)	Target Modern Architecture (To-Be)
API Route Binding	Legacy /api/scan, query string isolation	Versioned /api/v1/* schemas with typed Pydantic JSON bodies
Forensic State Storage	Local useState isolated inside each page	Global ForensicsProvider context / Zustand store persisting across tabs
WebSocket Subsystem	3 redundant connections; drops memory map	Single managed WebSocket singleton multiplexing alerts & memory maps
Theme & Palette	Warm parchment light mode; 'lime' is orange	CrowdStrike deep navy (#0A0E1A / #C6FF3D) with dark/light CSS variables
UI Primitives	Inline copy-pasted Tailwind utility classes	Shared UI primitives (Button, Badge, Card, Modal, CodeBox)
Kernel Telemetry Feed	Simulated ambient strings on setInterval	Pure live backend event feed driven exclusively by WebSocket frames

7. Prioritized, Staged Modernization Roadmap

Stage 1: Quick Wins & Backend Alignment (Low Effort — 1 to 2 Days)

• **Upgrade API Client to /api/v1:** Update src/lib/api.ts to consume /api/v1/scan, /api/v1/audit, and /api/v1/isolate/{pid} passing JSON bodies.
• **Consolidate WebSocket Connections:** Deprecate ad-hoc calls in pages; manage a single persistent socket inside ConnectionContext with event listener dispatching.
• **Retire Artificial Tickers:** Remove the ambient setInterval fake log generator in app/page.tsx and kernel-log/page.tsx.
• **Color Token Rectification:** Rename the orange token to brandOrange and restore lime to authentic SOC neon-green (#C6FF3D).

Stage 2: State Unification & Design System Primitives (Medium Effort — 3 to 4 Days)

• **Implement Global Forensics Store:** Build ForensicsContext to hold active process scans, summary metrics, and isolated PIDs so navigation preserves analysis state.
• **Wire Live Memory Map Feed:** Update the WebSocket listener to parse type === 'memory_map' frames and project real virtual address blocks onto the Live Process Map.
• **Extract Shared UI Primitives:** Create reusable components for Button, Card, StatusBadge, DataGrid, and Modal to eliminate code duplication.
• **Full Dark / SOC Theme Support:** Restore the signature #0A0E1A palette as the default dark mode with seamless light/dark toggling via CSS custom properties.

Stage 3: Advanced Capabilities & Feature Panels (Large Effort — 4 to 6 Days)

• **Build Docker Sandbox Panel:** Implement src/app/sandbox/page.tsx supporting binary submission, job UUID tracking, polling, and execution logs.
• **Build Supply Chain Scanner Panel:** Implement src/app/supply-chain/page.tsx displaying live process DLL tables, Authenticode certificates, and SHA-256 copy helpers.
• **Accessible Radar Navigation:** Pair the HTML5 canvas radar on /live-map with an accessible, keyboard-focusable DOM tree to ensure full keyboard navigation.

8. Final Audit Sign-Off

AUDIT VERDICT & CONCLUSION:
The ShadowMem Next.js frontend has a clean build and static deployment pipeline, but suffers from significant architectural drift relative to the newly modernized backend. By following the staged 3-phase roadmap—aligning REST/WS protocols to /api/v1, unifying forensic state across routes, and implementing the proposed Sandbox and Supply Chain feature structures—the frontend can achieve true enterprise-grade SOC tool operational readiness.